Retrieved but Ignored

Where LLMs Lose Code Reuse When Refactoring

Evgenii Lialin Roman Vikhman

Project 10: Code Reuse / Refactoring with AI

Motivation

- Code assistants mostly **edit existing repositories**, not empty files.
- Known weakness: they **re-implement** logic the project already has.
  - AI-generated pull requests contain more redundant code than human ones [Huang et al. 2026, *More Code, Less Reuse*]
  - Models often re-implement available repository dependencies instead of calling them [Hai et al., RepoExec, NAACL Findings 2025]
- Duplicated code passes tests, but it is a maintenance cost.

Research question

When a model refactors a function that duplicates an existing helper, does it replace the duplicate with a call, and if not, **where is the reuse opportunity lost**?

The papers we build on (1/3): the reuse problem

More Code, Less Reuse [Huang et al., preprint 2026]

Empirical study of AI-generated pull requests: more redundant code, less reuse. → **our motivation.**

RepoExec [Hai et al., Findings of NAACL 2025]

Repository-level generation; introduces the *dependency invocation rate*. → **we adopt "does the model call the existing function" as the metric, but for refactoring.**

DependEval [Du et al., Findings of ACL 2025]

Benchmarks LLM understanding of repository dependencies. → **dependency awareness is a known weak spot.**

The papers we build on (2/3): refactoring benchmarks

SWE-Refactor [Xu et al., preprint 2026] real-world Java refactorings, repository level

MANTRA [Xu et al., preprint 2025] method-level refactoring with contextual RAG + multi-agent LLMs

RefactorBench [Gautam et al., ICLR 2025] multi-file Python refactorings for agents

CodeTaste [Thillen et al., ICML 2026] do models choose human-like refactorings?

Gap

They measure *correctness* or *agreement with humans*, not whether existing code is **reused**. None explains *why* reuse fails.

The papers we build on (3/3): retrieval, instructions, context

CodeRAG	[Zhang et al., EMNLP 2025] retrieval for repository-level completion
ReflectCode	[Liu et al., Findings of EMNLP 2025] repository-aware code retrieval
CoRet	[Fehr et al., ACL 2025 short] retriever for code editing
Think-Search-Patch	[Xiong et al., EMNLP 2025 Industry] retrieval-augmented repair
BM25	[Robertson & Zaragoza 2009] our retrieval baseline
Lost in the Middle	[Liu et al., TACL 2024] position of information in long contexts → **our position ablation**
CodeIF	[Yan et al., ACL 2025 Industry] instruction following in code generation → **generic vs. explicit reuse instruction**

Retrieval papers assume "retrieve it and the model will use it". We test that assumption.

Benchmark: tasks built by inlining

Original (reference)

```
def _strip(line):
    return line.rstrip("\n").strip()

def parse(lines):
    out = []
    for line in lines:
        out.append(_strip(line))
    return out
```

Task: helper inlined into the caller

```
def parse(lines):
    out = []
    for line in lines:
        out.append(line.rstrip("\n").strip())
    return out
```

(illustrative example)

- Kept only if the call site is **covered by tests** and the suite still passes.
- **129 tasks** from **12 Python libraries** (click, pygments, sqlparse, toolz, . . .); 36 cross-module; helpers 2–30 lines (median 7).
- Model succeeds if its refactoring **calls the helper** and the tests pass.

Experimental design

Context conditions

- NONE: function only
- BM25: top-5 retrieved functions
- ORACLE: helper + 4 distractors
- FILE: the whole module

Instructions

- *generic*: "improve readability and maintainability"
- *reuse*: + "reuse existing functions where possible"

Models (open, one 24 GB GPU)

- Qwen2.5-Coder-7B, DeepSeek-Coder-6.7B
- Qwen3-Coder-30B-A3B (4-bit)
- Qwen3.8-27B (4-bit, thinking off)

Reuse funnel: each failure goes to exactly one stage

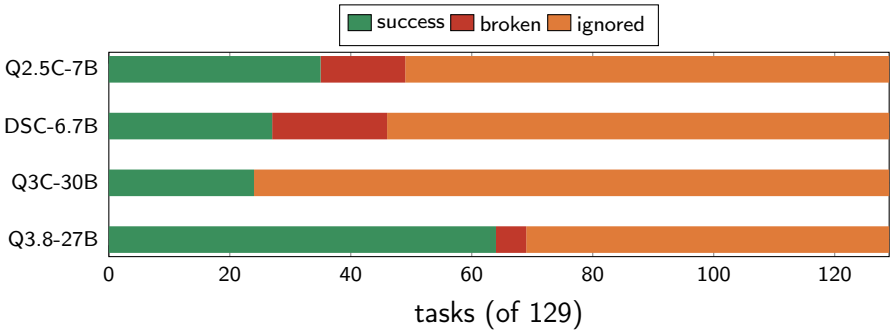
- **not in context** (retrieval)
- **ignored** (decision)
- **broken** (edit fails tests)

Results: helper reuse (% of 129 tasks)

Model	Instr.	NONE	BM25	ORACLE	FILE
Qwen2.5-Coder-7B	generic	1	21	38	18
	reuse	3	43	53	22
DeepSeek-Coder-6.7B	generic	4	33	36	18
	reuse	5	47	45	19
Qwen3-Coder-30B	generic	2	22	19	9
	reuse	5	41	54	27
Qwen3.8-27B	generic	5	50	53	36
	reuse	12	**74**	**84**	64

- Without context, models re-implement (reuse $\leq 12\%$).
- The explicit reuse instruction helps in every condition (up to $2.8\times$).
- Qwen3-Coder writes valid code far more often (83–93% vs. 36–60%) but does **not** reuse more.

The decision, not retrieval, is the main loss



ORACLE context, generic instruction: the helper is *always* in the prompt.

- Three of four models ignore the helper in **62–81%** of tasks.
- BM25 misses the helper in only 24% (recall@5 = 76%, MRR = 0.58).
- Better retrieval alone would recover few tasks.

Two controls

Position [after Lost in the Middle]

Same oracle snippets, helper first vs. last:

- DeepSeek (reuse instr.): 75 vs. 55 tasks, $p < .001$
- Qwen3-Coder (generic): 37 vs. 25, $p = .004$
- Qwen3.8 (generic): 78 vs. 69, $p = .035$
- Reordering alone changes up to 30 task outcomes (23%).

Renaming (memorisation check)

Helper shown as `fn_<hash>`; 114 tasks:

- Qwen3.8, BM25 generic: 56 $\rightarrow$ 22
- Qwen3.8, ORACLE reuse: 96 $\rightarrow$ 82
- Original name recalled from memory: only 18 / 912 and 5 / 912 outputs
- $\Rightarrow$ names act as a **cue**; direct memorisation is rare.

Conclusions

- Reuse is lost mainly at the model's **decision**: it sees the helper and ignores it.
- An explicit reuse instruction is the cheapest fix; only the newest model reuses well.
- Higher code quality $\neq$ more reuse.
- Reuse depends on surface cues: the helper's position and name.

Limitations and next steps

- Inlined code is a near-verbatim copy; next: tasks mined from real commits, paraphrased duplicates.
- Python only, open models only; dense and agentic retrieval not yet evaluated.

References

- Du et al. 2025. DependEval: Benchmarking LLMs for Repository Dependency Understanding. Findings of ACL 2025.
- Fehr et al. 2025. CoRet: Improved Retriever for Code Editing. ACL 2025 (short).
- Gautam et al. 2025. RefactorBench: Evaluating Stateful Reasoning in Language Agents Through Code. ICLR 2025.
- Hai et al. 2025. On the Impacts of Contexts on Repository-Level Code Generation (RepoExec). Findings of NAACL 2025.
- Huang et al. 2026. More Code, Less Reuse: Investigating Code Quality and Reviewer Sentiment towards AI-generated Pull Requests. arXiv:2601.21276.
- Liu et al. 2024. Lost in the Middle: How Language Models Use Long Contexts. TACL.
- Liu et al. 2025. Beyond Function-Level Search: Repository-Aware Dual-Encoder Code Retrieval with Adversarial Verification. Findings of EMNLP 2025.
- Robertson & Zaragoza 2009. The Probabilistic Relevance Framework: BM25 and Beyond. FnTIR.
- Thillen et al. 2026. CodeTaste: Can LLMs Generate Human-Level Code Refactorings? ICML 2026.
- Xiong et al. 2025. Think-Search-Patch: A Retrieval-Augmented Reasoning Framework for Repository-Level Code Repair. EMNLP 2025 Industry.
- Xu et al. 2025. MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration. arXiv.
- Xu et al. 2026. SWE-Refactor: A Repository-Level Benchmark for Real-World LLM-Based Code Refactoring. arXiv.
- Yan et al. 2025. CodeIF: Benchmarking the Instruction-Following Capabilities of LLMs for Code Generation. ACL 2025 Industry.
- Zhang et al. 2025. CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion. EMNLP 2025.
- Models: Qwen2.5-Coder (Hui et al. 2024), DeepSeek-Coder (Guo et al. 2024), Qwen3 (Yang et al. 2025), Qwen3.8; serving: vLLM (Kwon et al. 2023).